Frontend Interview Prep — Round 2

50 Questions & Answers · React · Next.js · HTML · CSS · TypeScript · Design Systems

Prepared for: Ali — Role-Related Knowledge (hands-on frontend)

Focus: Next.js 16 App Router · React 19 · write & debug code

How to use this: Cover the answer, say yours out loud, then compare. The interview is hands-on — practice *speaking* the answer and writing tiny code snippets, not just reading. Sections: React (1–10), Next.js (11–27), HTML (28–35), CSS (36–45), TypeScript (46–48), Design Systems (49–50).

Part 1 — React Fundamentals (Q1–Q10)

The hands-on round always probes these. Know them cold.

Q1. What is a React component re-render, and what triggers it?

A re-render is React **calling your component function again** to produce new JSX, then diffing it against the previous output and updating only the changed DOM nodes. **Re-render ≠ DOM update.** Triggers: (1) its own **state changes** via `setState`; (2) its **parent re-renders**; (3) its **props or consumed context change**.

> **Say this:** "Re-render means React re-runs the function; it only touches the DOM where the diff differs."

Q2. Explain the `useEffect` dependency array: `[]` vs `[x]` vs no array.

`useEffect(fn)` (no array) → runs **after every render** (most common cause of infinite loops). `useEffect(fn, [])` → runs **once after mount** (cleanup on unmount). `useEffect(fn, [x])` → runs on mount and whenever `x` changes. The effect runs **after** the render is committed/painted.

> **Trap:** setState inside an effect with no deps array → re-render → effect runs again → infinite loop.

Q3. Why does React need the `key` prop, and why is `key={id}` better than `key={index}`?

A `key` is a **stable, unique identifier** React uses to track list items across re-renders during reconciliation — to know which items were added, removed, or moved, so it can reuse DOM nodes and component state. `key={index}` breaks when the list reorders/deletes: indexes shift, so React mismatches rows, causing wrong updates and lost input state. A stable `id` follows the item.

> **Note:** `key` is a special React prop — not an HTML `id`. They are unrelated.

Q4. What is "derived state" and why avoid storing it in `useState`?

Derived data is anything computable from existing props/state (e.g. a filtered list from `items + query`). **Compute it during render** instead of storing in state + syncing via `useEffect`. Storing it duplicates the source of truth, causes bugs and extra renders. Rule: *"Don't store what you can derive."* Use `useMemo` only if the computation is expensive.

```
const results = items.filter(i ⇒ i.name.includes(query)); // not useState
```

Q5. Difference between controlled and uncontrolled components?

A **controlled** input's value is driven by React state (`value={x} onChange={...}`) — React is the single source of truth. An **uncontrolled** input keeps its own DOM state, read via a `ref` when needed. Controlled is preferred for validation/live UI; uncontrolled is simpler for basic forms.

Q6. What problem do `useMemo` and `useCallback` solve, and when should you NOT use them?

`useMemo` caches a **computed value**; `useCallback` caches a **function reference** across renders. Use them to avoid expensive recalculations or to keep stable references for memoized children / effect deps. Don't sprinkle them everywhere — premature memoization adds complexity and memory cost without benefit. Measure first.

Q7. What are the Rules of Hooks?

(1) Only call hooks at the **top level** — never inside loops, conditions, or nested functions. (2) Only call hooks from **React function components or custom hooks**. React relies on consistent **call order** across renders to match each hook to its state.

Q8. What is a custom hook and when would you write one?

A custom hook is a function starting with `use` that **reuses stateful logic** by composing built-in hooks (e.g. `useDebounce`, `useLocalStorage`). It shares *logic*, not state — each call gets its own isolated state. Write one when the same hook logic appears in multiple components.

Q9. How does state batching work, and why might two `setState` calls not double-update?

React **batches** multiple state updates in the same event into a single re-render for performance. Because state is a **snapshot** per render, `setCount(count+1); setCount(count+1)` both read the same `count`. Use the **updater function** `setCount(c ⇒ c+1)` to queue updates based on the latest value.

Q10. Lifting state up vs Context vs a state library — when to use each?

Lift state up to the closest common parent when a few nearby components share it. Use **Context** for low-frequency global values (theme, current user, locale) to avoid prop drilling. Reach for a library (Zustand, Redux, TanStack Query) for complex/global client state or server-cache state. Don't put fast-changing values in Context (causes wide re-renders).

Part 2 — Next.js (App Router) (Q11–Q27)

Heaviest section — they asked you to build with Next, so expect deep questions here.

Q11. Server Components vs Client Components — the core rule?

In the App Router, components are **Server Components by default**: they run only on the server, their code is **never shipped to the browser bundle**, and they can't use hooks/state/browser APIs. You opt into a **Client Component** with `"use client"` at the top of the file — only when you need **interactivity**: state, effects, event handlers, or browser APIs.

> **Rule:** Server by default; `"use client"` only for state / events / browser APIs.

Q12. Why can a Server Component be `async` and `await` data directly?

Because it runs **once on the server** during the request and produces HTML — there's no client re-render loop. React natively supports **async Server Components**: the server waits for the `await` to resolve, then streams the finished HTML. This replaces most `useEffect`-based data fetching.

```
async function Page() {
  const data = await getData(); // direct await, server only
  return <List items={data} />;
}
```

Q13. How do you fetch data and where should fetching live in the App Router?

Fetch **directly inside async Server Components** (or in a data-access layer they call). Next.js extends `fetch` with caching/revalidation. Keep data fetching on the server close to where it's used; only fetch on the client for truly interactive/real-time needs. This reduces bundle size and avoids client waterfalls.

Q14. Explain rendering strategies: SSR, SSG, ISR, and CSR.

SSR: HTML rendered per request (fresh, dynamic). **SSG**: HTML built once at build time (fastest, static). **ISR** (Incremental Static Regeneration): static pages regenerated in the background after a revalidate interval. **CSR**: rendered in the browser via JS. App Router lets you choose per route via caching/ `revalidate` / `dynamic` settings.

Q15. What does `loading.tsx` do and what powers it?

It's an **automatic Suspense fallback** for a route segment. Next.js wraps the page in `<Suspense>`; while the async Server Component awaits data, `loading.tsx` **streams to the browser instantly**, then the finished UI streams in and replaces it. Powered by **React Suspense + streaming SSR**.

Q16. What is the special file convention? (`layout`, `page`, `loading`, `error`, `not-found`, `route`)

`page.tsx` = the route UI; `layout.tsx` = shared wrapper that **persists** across child navigations (doesn't re-render); `loading.tsx` = Suspense fallback; `error.tsx` = error boundary (must be a Client Component); `not-found.tsx` = 404 UI; `route.ts` = API endpoint (Route Handler); `template.tsx` = like layout but re-mounts on navigation.

Q17. How does file-based routing and dynamic routing work?

Folders define URL segments; `page.tsx` makes a segment routable. **Dynamic segments** use brackets: `app/servers/[id]/page.tsx` → `/servers/srv-1`, with `params.id` available. `[...slug]` is catch-all, `[[...slug]]` optional catch-all. Route groups `(group)` organize folders without affecting the URL (you used `(auth)` and `(dashboard)`).

Q18. What does Next.js `<Link>` do that a plain `<a>` doesn't?

It performs **client-side (soft) navigation**: fetches only the new route's RSC payload, keeps unchanged layouts mounted, updates the URL — **no full page reload**. It also **prefetches** linked routes in the viewport for near-instant transitions. A plain `<a>` triggers a full document reload.

Q19. Why store filter/sort state in the URL instead of `useState`?

URL state is **shareable/bookmarkable** (send a teammate `?status=Down` and they see the same view), **survives refresh**, works with **Back/Forward** buttons, and is **server-renderable** on first load. `useState` resets on refresh and isn't shareable. Phrase: *"URL as the single source of truth for shareable, refresh-safe UI state."*

Q20. What is `middleware.ts` and is it enough for auth?

Middleware runs **before a request completes** (at the edge) — great for redirects, auth gating, rewrites, headers. It's a **UX/optimization gate, not your only security layer.** You must **re-check the session at the data layer** (Server Component / Route Handler), because middleware matchers can have gaps and have historically had bypass CVEs. Authorization belongs next to the data.

Q21. Server Actions — what are they and when to use them?

Server Actions are `"use server"` **async functions that run on the server** but can be called from client/forms without manually writing an API route. Use them for mutations (form submit, create/update/delete). They integrate with progressive enhancement and let you `revalidatePath` / `revalidateTag` after a change.

Q22. How does caching & revalidation work in the App Router?

Next.js can cache fetch results and rendered routes. Control freshness with `fetch(url, { next: { revalidate: 60 } })` (time-based ISR), `{ cache: 'no-store' }` (always dynamic), or tag-based `revalidateTag()` / `revalidatePath()` after mutations. Note caching defaults have changed across Next versions — **check the docs in `node_modules/next/dist/docs/` for your exact version.**

Q23. How do Route Handlers (`route.ts`) differ from pages, and how did your auth use them?

`route.ts` exports HTTP methods (`GET` , `POST` , ...) and returns a `Response` — it's your API layer (no UI). Your app uses `api/login` , `api/signup` , `api/logout` handlers to verify credentials, sign a JWT, and set/clear the **HttpOnly session cookie** via response headers.

Q24. How do you read `searchParams` and `params` , and why validate them?

In Server Components they're provided as props (in recent Next versions `searchParams` / `params` are **awaited Promises**: `const { id } = await params;`). Always **validate** against an allow-list — they're **untrusted user input** and TS types are erased at runtime, so a bad value could break rendering or be an injection/XSS vector.

```
const status = allowed.includes(raw) ? raw : "All"; // allow-list
```

Q25. How do you handle environment variables in Next.js? What does `NEXT_PUBLIC_` mean?

Server-only secrets (like `AUTH_SECRET` , `DATABASE_URL`) stay on the server and are read via `process.env` in server code. Only variables prefixed with `NEXT_PUBLIC_` are inlined into the **browser bundle** — so never put secrets there. Store local values in `.env.local` (gitignored).

Q26. How does Next.js optimize images and fonts?

`next/image` gives automatic resizing, modern formats (WebP/AVIF), lazy loading, and prevents layout shift via required width/height. `next/font` self-hosts fonts at build time (no external request), supports subsetting, and avoids layout shift with `font-display` . Both improve Core Web Vitals (LCP, CLS).

Q27. What is hydration, and what causes a "hydration mismatch" error?

Hydration is React **attaching event listeners and state to server-rendered HTML** in the browser, making it interactive. A **mismatch** happens when the client's first render differs from the server's HTML — e.g. using `Date.now()`, `window`, random values, or `localStorage` during render. Fix by rendering such values in `useEffect` or gating with a mounted flag.

Part 3 — HTML (Q28–Q35)

Often used to screen for accessibility and fundamentals.

Q28. What is semantic HTML and why does it matter?

Semantic HTML uses elements that describe their **meaning** (`<header>` , `<nav>` , `<main>` , `<article>` , `<footer>` , `<button>`) rather than generic `<div>` / `<span>` . Benefits: **accessibility** (screen readers), **SEO**, and **maintainability**. Example: use `<button>` for actions — it's focusable and keyboard-operable for free.

Q29. What are the most important accessibility (a11y) practices?

Use semantic elements; provide `alt` **text** on images; label form inputs (`<label htmlFor>`); ensure **keyboard navigation** and visible **focus states**; maintain **color contrast** (WCAG AA 4.5:1); use **ARIA** only when semantics aren't enough; manage focus in modals. Test with keyboard-only and a screen reader.

Q30. Difference between `<div>` and `<span>` ? Block vs inline elements?

`<div>` is a **block** element (starts on a new line, takes full width); `<span>` is **inline** (flows within text, only as wide as content). Block elements stack vertically; inline elements sit side by side and ignore top/bottom width/height in the normal flow.

Q31. What does the `<!DOCTYPE html>` and `<meta viewport>` do?

`<!DOCTYPE html>` tells the browser to use **standards mode** (HTML5) instead of legacy quirks mode. `<meta name="viewport" content="width=device-width, initial-scale=1">` makes the page **responsive** by mapping the layout viewport to the device width — essential for mobile.

Q32. How do you build an accessible, semantic form?

Wrap in `<form>` ; pair every input with a `<label>` (via `htmlFor` / `id`); use correct `type` (`email` , `password`), `name` , `required` , and `autocomplete` ; use a real `<button type="submit">` ; show validation messages tied to inputs with `aria-describedby` . Native validation + semantics gives keyboard and screen-reader support for free.

Q33. What's the difference between `id` and `class` attributes?

`id` must be **unique** per page and targets a single element (also used for anchors and `label htmlFor`); `class` can be **reused** on many elements and is the primary styling hook. In CSS specificity, `id` (100) outweighs `class` (10) — prefer classes for styling to keep specificity low.

Q34. What are `data-*` attributes used for?

Custom `data-*` attributes store **extra information on elements** without inventing non-standard attributes, readable via `element.dataset` in JS or `[data-state="open"]` in CSS. Common for hooks, test selectors (`data-testid`), and styling state. Don't use them for large data or sensitive info.

Q35. Explain the DOM and event bubbling/delegation.

The **DOM** is the browser's tree representation of the HTML that JS can read/modify. **Bubbling**: an event fires on the target then propagates up to ancestors. **Delegation**: attach one listener on a parent and use `event.target` to handle many children — efficient for dynamic lists. Use `stopPropagation()` to stop bubbling, `preventDefault()` to cancel default behavior.

Part 4 — CSS (Q36–Q45)

You used Tailwind — expect both raw CSS concepts and utility-class reasoning.

Q36. Explain the CSS Box Model.

Every element is a box: **content** → **padding** → **border** → **margin** (inside out). `width` / `height` apply to content by default. `box-sizing: border-box` makes width include padding+border (far more predictable — most resets set this globally). Margins collapse vertically between block elements.

Q37. Flexbox vs Grid — when to use each?

Flexbox is one-dimensional (a row OR a column) — ideal for toolbars, nav bars, aligning items in a line. **Grid** is two-dimensional (rows AND columns) — ideal for page layouts and card grids. Rule of thumb: content-driven single axis → Flex; layout-driven 2D structure → Grid. They compose well together.

Q38. How does CSS specificity work and how is a conflict resolved?

Specificity ranks selectors: inline style (1000) > `id` (100) > class/attribute/pseudo-class (10) > element/pseudo-element (1). Higher specificity wins; if equal, the **later** rule wins (source order). `!important` overrides all (avoid it). Keep specificity low — prefer classes — so styles stay overridable.

Q39. Explain `position`: static, relative, absolute, fixed, sticky.

static: normal flow (default). **relative**: offset from its normal spot; creates a positioning context. **absolute**: removed from flow, positioned relative to nearest positioned ancestor. **fixed**: positioned relative to the viewport (stays on scroll). **sticky**: acts relative until a scroll threshold, then sticks (great for headers).

Q40. How do you make a layout responsive?

Mobile-first with **media queries** (`@media (min-width:768px)`), fluid units (`%` , `rem` , `fr` , `clamp()`), flexible **Flex/Grid** layouts, `max-width` on containers, responsive images, and the viewport meta tag. In Tailwind, use breakpoint prefixes like `sm:` `md:` `lg:` (you used `sm:flex-row`).

Q41. Difference between `rem` , `em` , `px` , `%` , and viewport units?

`px` = absolute. `em` = relative to the element's font-size (compounds with nesting). `rem` = relative to the **root** font-size (predictable — preferred for spacing/typography and accessibility). `%` = relative to the parent. `vw/vh` = relative to viewport width/height. Use `rem` for scalable, accessible sizing.

Q42. What are CSS custom properties (variables) and why use them?

Declared with `--name: value` and read via `var(--name)`; they **cascade and can change at runtime** (e.g. theming/dark mode by overriding on a parent). Unlike Sass variables (compile-time), CSS variables are live in the browser. Your app uses them: `bg-app-card`, `--on-accent`, `--row-hover` — the backbone of a themeable design system.

Q43. How do you implement dark mode in CSS / Tailwind?

Define color **tokens as CSS variables** and override them under a `.dark` class or `@media (prefers-color-scheme: dark)`; components reference the tokens, not raw colors. In Tailwind, use the `dark:` variant or a class strategy. This keeps one set of components that re-theme by swapping variable values.

Q44. What's the difference between `visibility:hidden`, `display:none`, and `opacity:0`?

`display:none` removes the element from layout entirely (no space, not focusable). `visibility:hidden` hides it but **keeps its space**. `opacity:0` makes it invisible but it still **occupies space and remains interactive** (clickable/focusable). Choose based on whether you need layout space and interactivity.

Q45. What is the utility-first approach (Tailwind) — pros and cons?

Pros: styles colocated with markup, no naming/dead-CSS problems, consistent spacing/scale via design tokens, fast iteration, small production CSS (unused classes purged). **Cons:** verbose class strings, learning curve, can hurt readability without componentization. Mitigate by extracting repeated patterns into components and using `@apply` or tokens for shared styles.

Part 5 — TypeScript (Q46–Q48)

Q46. What is a union / string literal type, and how does it differ from a TS `enum`?

`type Status = "Up" | "Down" | "Degraded"` is a **string literal union**: it restricts a value to specific allowed strings, gives autocomplete, and is **compile-time only** (zero runtime output). A TS `enum` generates real JavaScript at runtime. Modern codebases prefer unions for their zero-cost erasure.

Q47. `type` vs `interface` — what's the real difference?

Both can model objects. `interface` supports **declaration merging** and uses `extends` — idiomatic for object/class shapes and public APIs. `type` can express **unions, intersections, tuples, and primitive aliases** that interfaces can't (e.g. `ServerStatus` must be a `type` because it's a union).

Q48. Why validate untrusted input even with TypeScript? What are generics?

TS types are **erased at runtime** — a value cast as `ServerStatus` could actually be anything from a URL/API, so validate with an **allow-list** or schema (e.g. Zod). **Generics** let you write reusable, type-safe code parameterized by a type: `function first<T>(arr: T[]): T` preserves the element type for the caller.

Part 6 — Design Systems (Q49–Q50)

Q49. What is a design system and what are design tokens?

A **design system** is a shared, reusable set of **components, patterns, guidelines, and tokens** that keep a product consistent and faster to build (e.g. Material, shadcn/ui). **Design tokens** are the named primitives — colors, spacing, typography, radii, shadows (e.g. `--color-accent`, `--space-4`) — usually stored as CSS variables so themes (light/dark/brand) swap by changing token values, not components.

Q50. How do you build a reusable, accessible component (e.g. a Button) in a design system?

Design a clear **props API** (`variant`, `size`, `disabled`, `asChild`); base it on the correct **semantic element** (`<button>`); style via **tokens** (not hardcoded colors) so it themes automatically; ensure **accessibility** (focus ring, `aria-*`, keyboard support, disabled handling); keep it **composable** and documented (Storybook). Variants are best managed with a tool like `cva` (class-variance-authority) for consistent, type-safe class combinations.

> **Senior signal:** "Components consume tokens, never raw values — that's what makes the system themeable and consistent."

Generated for Ali · 50 Q&A · React · Next.js 16 (App Router) · HTML · CSS · TypeScript · Design Systems
Tip: For Next.js version-specific behavior (caching, params), always confirm against `node_modules/next/dist/docs/` in your project.